

Documentação Técnica e Arquitetura - MS Sustentável

Esta documentação tem o objetivo de detalhar o *Stack Tecnológico*, a arquitetura de módulos e as **Diretrizes de Segurança e Infraestrutura** da aplicação MS Sustentável.

Público Alvo: Desenvolvedores, Engenheiros de Segurança e DevOps responsáveis pelo deploy em nuvem.

1. Stack Tecnológico e Ferramentas

O projeto foi construído sob uma arquitetura serverless moderna, separando o frontend (SPA) do backend as a service (BaaS).

1.1 Frontend

- **React 19:** Biblioteca base para construção das interfaces do usuário. Utilizado com `React Hooks` para gerência de estado local.
- **TypeScript:** Superset de tipagem forte para JavaScript, prevenindo erros de tipagem e garantindo contratos de dados consistentes com o banco de dados.
- **Vite 8:** Build tool incrivelmente rápida para ambiente de desenvolvimento local e bundling de produção (rollup).
- **Tailwind CSS 4:** Framework utilitário de CSS usado para construir layouts de forma rápida, criando designs responsivos sem a necessidade de arquivos CSS separados (exceto regras customizadas avançadas no `index.css`).
- **Lucide React:** Biblioteca vetorial SVG para os ícones fluídos e leves de toda a interface.
- **MapLibre GL & React Map GL:** Engine de renderização de mapas vetoriais de código aberto. Utilizada para renderizar os polígonos e mapeamentos de fazendas em satélite/rua.
- **JSZip:** Biblioteca utilizada no frontend para descompactar arquivos complexos como Shapefiles, permitindo processamento local de dados do Sigef.

1.2 Backend & Infraestrutura (Supabase)

O projeto depende integralmente do **Supabase**, que atua como BaaS, provendo os seguintes serviços estruturais sobre PostgreSQL:

- **Supabase Auth:** Sistema de autenticação (JWT) para login seguro dos usuários, garantindo perfis (`produtor`, `tecnico`, `gestor`).
- **PostgreSQL com PostGIS:** Banco de dados relacional. PostGIS é usado para consultas geoespaciais, armazenamento de polígonos `geometry(Polygon, 4326)` de mapas do CAR/SIGEF.
- **Row Level Security (RLS):** Políticas de segurança a nível de linha aplicadas DIRETAMENTE no banco de dados. Evita que um usuário consulte, insira ou altere dados aos quais não tem permissão.
- **Supabase Storage:** Buckets seguros para armazenamento de imagens de perfil, PDFs anexados aos relatórios, fotos de evidências in loco e laudos.

1.3 Inteligência Artificial

- **Google Generative AI (Gemini):** Integrado diretamente via API (`@google/generative-ai`) para prover o "Briefing Executivo AI". Atua recebendo dados brutos de propriedades e devolvendo um resumo textual formatado dos gargalos da propriedade.

2. Estrutura de Autenticação e Autorização

A segurança da plataforma depende estritamente das Políticas do Supabase. O Frontend é 100% "cego" em relação à segurança. Ele apenas encaminha o JWT gerado pelo Supabase na conexão:

1. **Sessão JWT:** Quando o usuário loga via `supabase.auth.signInWithPassword`, um token JWT é salvo de maneira segura.
2. **Contexto de Autenticação (`AuthContext.tsx`):** O Frontend intercepta mudanças de sessão e bloqueia páginas (rotas protegidas no `App.tsx`) se não houver um `session.user`.
3. **Role Base Access Control (RBAC):** Ao lado do usuário (tabela `auth.users`), existe a tabela `perfis`. Ela contém o `role` (Papel). O RLS cruza o token do usuário com essa tabela para determinar as permissões.

Modos de Visualização e Regras

- **Produtor:** Só enxerga as próprias propriedades associadas. Não vê propriedades de terceiros (garantido por RLS `WHERE produtor_id = auth.uid()`).
- **Técnico:** Visualiza apenas as Auditorias e propriedades a ele designadas (`WHERE tecnico_responsavel_id = auth.uid()`).
- **Gestor:** Acesso `all` a todos os painéis, com RLS de leitura universal ou leitura `bypass` baseado em `role 'gestor'`.

3. Guia para o Especialista de Segurança (Antes do Deploy)

Para o time que irá subir a infraestrutura na Nuvem e refinar a segurança, os seguintes tópicos **exigem revisão metódica**:

3.1. Variáveis de Ambiente e Chaves (.env)

A aplicação consome três chaves primárias que **NÃO DEVEM NUNCA** ser expostas (com exceção das chaves `anon` públicas configuradas para o Vite):

- `VITE_SUPABASE_URL`: URL pública da instância Supabase.
- `VITE_SUPABASE_ANON_KEY`: Chave pública do Supabase. Essa chave é enviada no navegador cliente, mas só tem privilégios básicos que são barrados pelo RLS.
- `VITE_GEMINI_API_KEY`: Chave da API do Google Gemini. **ALERTA CRÍTICO**: Se a chamada para a IA está sendo feita do lado do Frontend, a chave do Gemini corre risco de exposição no navegador. *Ação Corretiva Recomendada antes da Nuvem*: Mover as chamadas da IA para uma Supabase Edge Function ou Node Serverless interno que esconda a chave do Google.

3.2. Políticas Row Level Security (RLS)

Antes de desativar o ambiente de desenvolvimento:

- Audite todas as tabelas no Supabase (propriedades, auditorias, pendências, documentos).
- Certifique-se de que nenhuma tabela tem `Enable RLS: Off` ou políticas em branco (pois permitiria CRUD anônimo mundial).
- Revise as *Migrations SQL* do repositório para garantir que os grants estão isolados corretamente.

3.3. Políticas de Storage

- Audite os buckets do Supabase Storage (`documentos_produtores`, etc).
- Documentos de fazendas são dados sensíveis. O Bucket **NÃO** pode ser marcado como `Public`. O Frontend deve requisitar `Signed URLs` (URLs assinadas com validade temporal curta) para baixar/renderizar imagens e PDFs.

3.4. CORS e Headers

- Na interface Web do Supabase (Nuvem), configure explicitamente as origens CORS permitidas. Exemplo: `https://app.mssustentavel.gov.br`. Remova `localhost` e `*` da lista de origens em Produção.

4. Considerações de Organização e LGPD

- Toda a estrutura de pastas criadas em `docs/` deve ser utilizada para compilar guias de usuário (que devem ser escritos com linguagem acessível ao produtor) e novas features técnicas.
- Devido à captura do **CAR (Cadastro Ambiental Rural)** e possivelmente CPF dos Produtores, a base de dados da aplicação se enquadra na LGPD. O Supabase Encryption pode ser configurado para proteger os dados em repouso. O Frontend já usa HTTPS garantido pelo provedor (seja Vercel, Netlify ou nuvem própria).

Fim do Documento.